Ideation onto basic product development

You can treat your F1 telemetry API as a small REST service with one main **telemetry_sample** model that exposes full Create, Read, Update, and Delete endpoints against a database, plus optional read-only analytics on top.[1]

Core data model

Design a single telemetry_sample entity that all CRUD operations target.[1]

Example fields:[1]

- id (integer primary key)
- session_id (e.g. "2025-AUS-RACE")
- driver_code (e.g. "VER")
- lap_number (integer)
- sample_time_ms (float, ms from lap start)
- speed_kph (float)
- rpm (integer)
- gear (integer, e.g. -1..8)
- throttle_pct (float 0–100)
- brake (bool)
- drs_active (bool)

This becomes your DB table and your JSON schema for requests/responses.[1]

Mapping CRUD to HTTP

Each CRUD operation maps to one or more REST endpoints on telemetry_sample.[2][3]

Create (C)

Goal: allow clients to insert new telemetry samples into your database.[3]

Typical endpoints:

- POST /api/v1/telemetry
  - Body: one telemetry_sample (without id) as JSON.
  - Behaviour: validate fields, insert into DB, return created record or its id.
  - Status: 201 Created on success, 400 on validation errors.[4][3]
- POST /api/v1/telemetry/bulk
  - Body: array of telemetry_sample objects.
  - Behaviour: validate all, insert many rows in one transaction, return list of ids or created records.
  - Use case: uploading a lap or full session at once.[5][4]

If you also build an ingest endpoint that fetches from OpenF1/FastF1 and then writes into your DB, it is still using "Create" under the hood, but for the assignment the core “C” is the client sending JSON to your POST endpoint.[6]

Read (R)

Goal: allow flexible querying and retrieval of saved telemetry.[2][3]

Endpoints:

- GET /api/v1/telemetry
  - Query params: optional filters like session_id, driver_code, lap_number, limit/offset.
  - Behaviour: return a paginated list of telemetry_sample records matching filters.
  - Status: 200 OK.[5][2]
- GET /api/v1/telemetry/{id}
  - Behaviour: return a single telemetry_sample by id.
  - Status: 200 OK if found, 404 Not Found if missing.[5]

Optional analytics (read-only, not part of CRUD but nice for the F1 angle):

- GET /api/v1/telemetry/{session_id}/{driver_code}/lap/{lap_number}/summary

    - Returns derived values such as average_speed_kph or max_speed_kph for that lap, computed from underlying samples.[7]

Update (U)

Goal: allow correction of stored samples (e.g. wrong gear or throttle value).[3]

Endpoints:

- PUT /api/v1/telemetry/{id}
    - Body: full telemetry_sample object (complete replacement).
    - Behaviour: validate; if exists, update all mutable fields; if not, 404.
    - Status: 200 OK or 404 Not Found.[3][5]
- PATCH /api/v1/telemetry/{id} (optional)
    - Body: partial fields (e.g. just speed_kph and gear).
    - Behaviour: update only provided fields.
    - Status: 200 OK or 404.[3]

You can justify using only PUT in your report if PATCH feels like scope creep.[2]

Delete (D)

Goal: allow removing bad or test data.[3]

Endpoints:

- DELETE /api/v1/telemetry/{id}
    - Behaviour: delete a single row by id.
    - Status: 204 No Content on success, 404 if not found.[5][3]
- (Optional) DELETE /api/v1/telemetry
    - Query params: filters such as session_id or driver_code.
    - Behaviour: bulk delete matching records; use with care.
    - Status: 204 No Content.[2]

Behaviour and status codes

For each operation you should consistently:[5][3]

- Return 201 Created for successful POST that creates a resource.
- Return 200 OK for successful GET and PUT/PATCH returning a body.
- Return 204 No Content for successful DELETE.
- Return 400 Bad Request when validation fails (e.g. negative speed_kph or gear outside allowed range).
- Return 404 Not Found when the requested id does not exist.
- Return 500 Internal Server Error for unexpected DB issues (ideally logged, generic message to client).

Using a framework like FastAPI + SQLAlchemy + SQLite/Postgres gives you JSON by default and clean mapping from HTTP verbs to DB operations.[8][9][2][5]

How a client will "hit" your API

User interaction examples (no code detail, just behaviour):[4][5]

- Create: POST JSON to /api/v1/telemetry from curl/Postman/Swagger UI to insert one or many samples.
- Read: GET /api/v1/telemetry?session_id=2025-AUS-RACE&driver_code=VER to fetch all samples for a driver; GET /api/v1/telemetry/123 to fetch a specific sample.
- Update: PUT /api/v1/telemetry/123 with corrected JSON to fix a sample.
- Delete: DELETE /api/v1/telemetry/123 to remove a bad row.

These four flows are what you describe in your report as demonstrating full CRUD on your telemetry_sample data model linked to a database.[2][5]

CRUD elements overview

Aspect	Create	Read	Update	Delete
HTTP method	POST	GET	PUT (and optional PATCH)	DELETE
URL (main)	/api/v1/telemetry, /api/v1/telemetry/bulk	/api/v1/telemetry, /api/v1/telemetry/{id}	/api/v1/telemetry/{id}	/api/v1/telemetry/{id}

Request body	New sample(s) JSON	None	Full or partial sample JSON	None
Main use	Insert new telemetry rows	Query and retrieve saved telemetry	Correct stored telemetry	Remove bad or test telemetry
Key status	201 on success, 400 on validation error	200 on success, 404 if not found (by id)	200 on success, 400/404 on error	204 on success, 404 if not found

Would it help most if we next sketch the actual FastAPI route signatures (just the function definitions and Pydantic models), or do you first want to nail down the exact fields you'll include in telemetry_sample for the coursework?

⁂

lets be invative i don't think createing data is the best idea as it the data is already real so it dosnt needt to be create exists i need to take an invative way of apporaching this problem

You can keep the data "real" and still satisfy CRUD by treating your API as a telemetry **curation and analytics** layer on top of existing sources like OpenF1, not as a fake data generator.[19][20]

Big idea

Instead of "creating" telemetry values, your API's job is to:

- Ingest authoritative telemetry from upstream sources (OpenF1, FastF1, your CSV uploads) into your own database in a controlled way.[20][19]
- Let users curate, annotate, slice, and analyse that telemetry via full CRUD on your own domain objects (sessions, driver runs, laps, annotations, favourites, derived metrics), while the raw numbers remain grounded in real data.[21]

So "C" is not "invent numbers", it is "create a record in your system that references real telemetry".

Innovative data model options

Pick a higher-level entity than raw samples for CRUD, and keep samples read-only:

- session_dataset: “All telemetry for VER in AUS 2025 race” loaded from OpenF1 and stored once.
- lap_summary: one row per lap with averages, extremes, and gaps, computed from raw telemetry.[22][23]
- comparison_run: “HAM vs VER lap 12” storing config + cached comparison metrics (gap vs distance, sector losses, etc.).[24][23]
- annotation: user-added comments/labels on laps, corners, or intervals (“lockup”, “great exit”, etc.).[24]

Each of these can have full CRUD, even though the underlying telemetry stream is immutable.

How CRUD can look without inventing telemetry

Create

You can define “Create” as creating curated telemetry artefacts, not raw samples:

- POST /api/v1/sessions/import
  - Body: { source: "openf1", session_key: "2023, Bahrain, Race" }.
  - Behaviour: call OpenF1 /car_data, /laps, etc., pull real data, store it as your session_dataset row plus its associated samples/laps in your DB.[25][19][20]
  - This is creating a new representation of real data inside your system, not fabricating values.
- POST /api/v1/lap_summaries
  - Body: { session_id, driver_code, lap_number }.
  - Behaviour: compute summary metrics from stored samples (avg speed, max speed, time loss in sectors) and persist them as a new lap_summary row.[23][22]
- POST /api/v1/comparisons
  - Body: { session_id, driver_a, driver_b, lap_number }.
  - Behaviour: run a gap-vs-distance analysis and store the result as a comparison_run resource with metadata and maybe a pre-computed trace for plotting later.[23][24]
- POST /api/v1/annotations

    - Body: { session_id, driver_code, lap_number, start_distance_m, end_distance_m, note: "Tyre warmup issues" }.
    - Behaviour: attach user commentary to real telemetry intervals.[24]

In your report you can say: “Create operations never fabricate telemetry; they create curated resources (imports, summaries, comparisons, annotations) derived from or linked to real F1 data.”

Read

Read becomes very rich and clearly “real”:

- GET /api/v1/sessions/{id} → shows which real session you imported, its drivers and laps.[19][20]
- GET /api/v1/lap_summaries?session_id=…&driver_code=… → table of real lap metrics.[22][23]
- GET /api/v1/comparisons/{id} → returns the stored comparison_run for visualisation.[24]
- GET /api/v1/annotations?session_id=… → overlays user notes on real telemetry.[24]

Update

You don’t update raw telemetry, you update your curation layer:

- PUT/PATCH /api/v1/sessions/{id}
    - Change local labels (e.g. “Bahrain 2025 Race – Verstappen data only”) or visibility.
- PUT/PATCH /api/v1/annotations/{id}
    - Fix or expand a note about an oversteer moment or tyre phase.
- PUT/PATCH /api/v1/comparisons/{id}
    - Adjust options (alignment by time vs by distance, smoothing settings) and re-compute in the background.[23][24]

Delete

Delete also operates at the curated level:

- DELETE /api/v1/sessions/{id} → drop that imported session (and related summaries/comparisons) from your DB.
- DELETE /api/v1/annotations/{id} → remove a user note.
- DELETE /api/v1/comparisons/{id} → remove a stored comparison you no longer need.

Again, raw F1 data is authoritative; you're only removing your copy or your added layers.

Two concrete design patterns

You could frame the assignment using one of these patterns:

1. Telemetry import & summary API
    - Core model for CRUD: lap_summary.
    - Create: compute & store new lap summaries for a driver's laps after importing real telemetry.
    - Read: filter lap summaries for sessions/drivers; get one by id.
    - Update: correct labels/metadata on a summary (e.g. "push lap", "cooling lap").
    - Delete: remove summaries you don't want to keep.
2. Driver comparison API
    - Core model for CRUD: comparison_run.
    - Create: define a comparison between two real laps and compute gap-vs-distance + key metrics, store it.
    - Read: list and fetch comparisons, maybe with filters by drivers or track.[24]
    - Update: tweak parameters (window size, alignment method) and re-generate.
    - Delete: remove old comparisons.

Both are clearly grounded in real telemetry and feel more "innovative" than just CRUD over a raw_sample table, while still satisfying the coursework requirement that you implement CRUD on at least one data model linked to a database.[26]

Which direction feels more exciting for you: a lap_summary/analytics-first API, a comparison_run/driver-vs-driver API, or an annotation-layer API where users mark up interesting telemetry moments?

⁂

let start with my main and go thorught the implmentation of the basic c r u d operatoins

Let's wire up a minimal FastAPI `main.py` that gives you working CRUD endpoints for a single telemetry-like model; you can then adapt the fields to lap summaries or comparisons.

1. Project skeleton

You can keep it in one file for now (e.g. `main.py`) with:

- DB setup (SQLAlchemy + SQLite).
- ORM model (TelemetrySample).
- Pydantic schemas (for create/update/read).
- FastAPI app + CRUD routes.[31][32]

2. Minimal main.py with CRUD

```
# main.py
from typing import List, Optional

from fastapi import FastAPI, Depends, HTTPException, status
from pydantic import BaseModel, Field
from sqlalchemy import Column, Integer, String, Float, Boolean, create_engine
from sqlalchemy.orm import sessionmaker, Session, declarative_base

# ---------- Database setup ----------
SQLALCHEMY_DATABASE_URL = "sqlite:///./telemetry.db"

engine = create_engine(
    SQLALCHEMY_DATABASE_URL, connect_args={"check_same_thread": False}
)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

Base = declarative_base()

# ---------- ORM model ----------
class TelemetrySample(Base):
    __tablename__ = "telemetry_samples"

    id = Column(Integer, primary_key=True, index=True)
    session_id = Column(String, index=True, nullable=False)
```

```
    driver_code = Column(String, index=True, nullable=False)
    lap_number = Column(Integer, nullable=False)
    sample_time_ms = Column(Float, nullable=False)
    speed_kph = Column(Float, nullable=False)
    rpm = Column(Integer, nullable=False)
    gear = Column(Integer, nullable=False)
    throttle_pct = Column(Float, nullable=False)
    brake = Column(Boolean, nullable=False)
    drs_active = Column(Boolean, nullable=False)

# create tables
Base.metadata.create_all(bind=engine)

# ---------- Pydantic schemas ----------
class TelemetrySampleBase(BaseModel):
    session_id: str = Field(..., example="2025-AUS-RACE")
    driver_code: str = Field(..., example="VER")
    lap_number: int = Field(..., ge=1, example=15)
    sample_time_ms: float = Field(..., ge=0, example=24567.3)
    speed_kph: float = Field(..., ge=0, example=285.4)
    rpm: int = Field(..., ge=0, example=11250)
    gear: int = Field(..., ge=-1, le=8, example=7)
    throttle_pct: float = Field(..., ge=0, le=100, example=92.3)
    brake: bool = Field(..., example=True)
    drs_active: bool = Field(..., example=True)

class TelemetrySampleCreate(TelemetrySampleBase):
    pass  # same fields as base

class TelemetrySampleUpdate(BaseModel):
    # All optional for PATCH-style updates
    session_id: Optional[str] = None
    driver_code: Optional[str] = None
    lap_number: Optional[int] = Field(None, ge=1)
    sample_time_ms: Optional[float] = Field(None, ge=0)
    speed_kph: Optional[float] = Field(None, ge=0)
    rpm: Optional[int] = Field(None, ge=0)
    gear: Optional[int] = Field(None, ge=-1, le=8)
    throttle_pct: Optional[float] = Field(None, ge=0, le=100)
    brake: Optional[bool] = None
    drs_active: Optional[bool] = None

class TelemetrySampleRead(TelemetrySampleBase):
    id: int
```

```
    class Config:
        orm_mode = True  # allow returning ORM objects

# ---------- FastAPI app ----------
app = FastAPI(title="F1 Telemetry CRUD API")

# Dependency: DB session per request
def get_db() -> Session:
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

# ---------- CRUD endpoints ----------

# Create: single sample
@app.post(
    "/api/v1/telemetry",
    response_model=TelemetrySampleRead,
    status_code=status.HTTP_201_CREATED,
)
def create_telemetry_sample(
    telemetry_in: TelemetrySampleCreate, db: Session = Depends(get_db)
):
    sample = TelemetrySample(**telemetry_in.dict())
    db.add(sample)
    db.commit()
    db.refresh(sample)
    return sample

# Read: list with filters (R)
@app.get("/api/v1/telemetry", response_model=List[TelemetrySampleRead])
def list_telemetry_samples(
    session_id: Optional[str] = None,
    driver_code: Optional[str] = None,
    lap_number: Optional[int] = None,
    db: Session = Depends(get_db),
):
    query = db.query(TelemetrySample)
    if session_id is not None:
        query = query.filter(TelemetrySample.session_id == session_id)
    if driver_code is not None:
```

```
        query = query.filter(TelemetrySample.driver_code == driver_code)
    if lap_number is not None:
        query = query.filter(TelemetrySample.lap_number == lap_number)
    return query.all()

# Read: single by id (R)
@app.get("/api/v1/telemetry/{sample_id}", response_model=TelemetrySampleRead)
def get_telemetry_sample(sample_id: int, db: Session = Depends(get_db)):
    sample = db.query(TelemetrySample).get(sample_id)
    if not sample:
        raise HTTPException(status_code=404, detail="Telemetry sample not found")
    return sample

# Update: partial fields (U)
@app.patch("/api/v1/telemetry/{sample_id}", response_model=TelemetrySampleRead)
def update_telemetry_sample(
    sample_id: int,
    telemetry_update: TelemetrySampleUpdate,
    db: Session = Depends(get_db),
):
    sample = db.query(TelemetrySample).get(sample_id)
    if not sample:
        raise HTTPException(status_code=404, detail="Telemetry sample not found")

    update_data = telemetry_update.dict(exclude_unset=True)
    for field, value in update_data.items():
        setattr(sample, field, value)

    db.commit()
    db.refresh(sample)
    return sample

# Delete: by id (D)
@app.delete("/api/v1/telemetry/{sample_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_telemetry_sample(sample_id: int, db: Session = Depends(get_db)):
    sample = db.query(TelemetrySample).get(sample_id)
    if not sample:
        raise HTTPException(status_code=404, detail="Telemetry sample not found")
    db.delete(sample)
    db.commit()
    return None
```

This follows the standard FastAPI + SQLAlchemy CRUD pattern (models, schemas, dependency-injected DB session, and HTTP verbs mapped to DB actions).[33][32][31]

3. Running and interacting

- Start the app:

```
uvicorn main:app --reload
```

- Open the interactive docs at `http://127.0.0.1:8000/docs` to:
  - POST a telemetry sample (C).
  - GET list or single sample (R).
  - PATCH to change e.g. speed_kph (U).
  - DELETE by id (D).[34][35]

Once this is working, we can refactor the `TelemetrySample` into a more "innovative" model (e.g. `LapSummary` or `ComparisonRun`) while keeping the same CRUD structure.

Do you want to adapt this next to a lap_summary model (per lap aggregates) or to a comparison_run model (two drivers, one lap, with gap metrics) before you start writing your report?

⁂

i have seprate files for routers schemes and services

Perfect; then you want to slice that single-file example into `routers/`, `schemas/`, `services/`, plus the DB base in one place while keeping the same CRUD behaviour.[41][42]

Below is a clean structure you can drop into your project and then tweak to your innovative model later.

Project layout

- main.py
- db.py
- models.py
- schemas/telemetry.py

- services/telemetry.py
- routers/telemetry.py

You can rename `telemetry` later to `lap_summary` or `comparison`.

db.py

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base

SQLALCHEMY_DATABASE_URL = "sqlite:///./telemetry.db"

engine = create_engine(
    SQLALCHEMY_DATABASE_URL, connect_args={"check_same_thread": False}
)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

Base = declarative_base()

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

models.py

```
from sqlalchemy import Column, Integer, String, Float, Boolean
from db import Base

class TelemetrySample(Base):
    __tablename__ = "telemetry_samples"

    id = Column(Integer, primary_key=True, index=True)
    session_id = Column(String, index=True, nullable=False)
    driver_code = Column(String, index=True, nullable=False)
    lap_number = Column(Integer, nullable=False)
    sample_time_ms = Column(Float, nullable=False)
```

```
    speed_kph = Column(Float, nullable=False)
    rpm = Column(Integer, nullable=False)
    gear = Column(Integer, nullable=False)
    throttle_pct = Column(Float, nullable=False)
    brake = Column(Boolean, nullable=False)
    drs_active = Column(Boolean, nullable=False)
```

schemas/telemetry.py

```
from typing import Optional
from pydantic import BaseModel, Field

class TelemetrySampleBase(BaseModel):
    session_id: str = Field(..., example="2025-AUS-RACE")
    driver_code: str = Field(..., example="VER")
    lap_number: int = Field(..., ge=1)
    sample_time_ms: float = Field(..., ge=0)
    speed_kph: float = Field(..., ge=0)
    rpm: int = Field(..., ge=0)
    gear: int = Field(..., ge=-1, le=8)
    throttle_pct: float = Field(..., ge=0, le=100)
    brake: bool
    drs_active: bool

class TelemetrySampleCreate(TelemetrySampleBase):
    pass

class TelemetrySampleUpdate(BaseModel):
    session_id: Optional[str] = None
    driver_code: Optional[str] = None
    lap_number: Optional[int] = Field(None, ge=1)
    sample_time_ms: Optional[float] = Field(None, ge=0)
    speed_kph: Optional[float] = Field(None, ge=0)
    rpm: Optional[int] = Field(None, ge=0)
    gear: Optional[int] = Field(None, ge=-1, le=8)
    throttle_pct: Optional[float] = Field(None, ge=0, le=100)
    brake: Optional[bool] = None
    drs_active: Optional[bool] = None

class TelemetrySampleRead(TelemetrySampleBase):
    id: int
```

```
    class Config:
        orm_mode = True
```

services/telemetry.py

Business logic + DB ops live here so routers stay thin.[42][43]

```
from typing import List, Optional
from sqlalchemy.orm import Session
from models import TelemetrySample
from schemas.telemetry import (
    TelemetrySampleCreate,
    TelemetrySampleUpdate,
    TelemetrySampleRead,
)

def create_sample(db: Session, telemetry_in: TelemetrySampleCreate) -> TelemetrySample:
    sample = TelemetrySample(**telemetry_in.dict())
    db.add(sample)
    db.commit()
    db.refresh(sample)
    return sample

def list_samples(
    db: Session,
    session_id: Optional[str] = None,
    driver_code: Optional[str] = None,
    lap_number: Optional[int] = None,
) -> List[TelemetrySample]:
    query = db.query(TelemetrySample)
    if session_id is not None:
        query = query.filter(TelemetrySample.session_id == session_id)
    if driver_code is not None:
        query = query.filter(TelemetrySample.driver_code == driver_code)
    if lap_number is not None:
        query = query.filter(TelemetrySample.lap_number == lap_number)
    return query.all()

def get_sample(db: Session, sample_id: int) -> Optional[TelemetrySample]:
    return db.query(TelemetrySample).get(sample_id)

def update_sample(
```

```
    db: Session, sample_id: int, telemetry_update: TelemetrySampleUpdate
) -> Optional[TelemetrySample]:
    sample = db.query(TelemetrySample).get(sample_id)
    if not sample:
        return None
    data = telemetry_update.dict(exclude_unset=True)
    for field, value in data.items():
        setattr(sample, field, value)
    db.commit()
    db.refresh(sample)
    return sample

def delete_sample(db: Session, sample_id: int) -> bool:
    sample = db.query(TelemetrySample).get(sample_id)
    if not sample:
        return False
    db.delete(sample)
    db.commit()
    return True
```

routers/telemetry.py

```
from typing import List, Optional
from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.orm import Session

from db import get_db
from schemas.telemetry import (
    TelemetrySampleCreate,
    TelemetrySampleUpdate,
    TelemetrySampleRead,
)
from services import telemetry as telemetry_service

router = APIRouter(prefix="/api/v1/telemetry", tags=["telemetry"])

@router.post(
    "",
    response_model=TelemetrySampleRead,
    status_code=status.HTTP_201_CREATED,
)
def create_telemetry(
```

```
    telemetry_in: TelemetrySampleCreate,
    db: Session = Depends(get_db),
):
    return telemetry_service.create_sample(db, telemetry_in)

@router.get("", response_model=List[TelemetrySampleRead])
def list_telemetry(
    session_id: Optional[str] = None,
    driver_code: Optional[str] = None,
    lap_number: Optional[int] = None,
    db: Session = Depends(get_db),
):
    return telemetry_service.list_samples(db, session_id, driver_code, lap_number)

@router.get("/{sample_id}", response_model=TelemetrySampleRead)
def get_telemetry(sample_id: int, db: Session = Depends(get_db)):
    sample = telemetry_service.get_sample(db, sample_id)
    if not sample:
        raise HTTPException(status_code=404, detail="Telemetry sample not found")
    return sample

@router.patch("/{sample_id}", response_model=TelemetrySampleRead)
def update_telemetry(
    sample_id: int,
    telemetry_update: TelemetrySampleUpdate,
    db: Session = Depends(get_db),
):
    sample = telemetry_service.update_sample(db, sample_id, telemetry_update)
    if not sample:
        raise HTTPException(status_code=404, detail="Telemetry sample not found")
    return sample

@router.delete("/{sample_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_telemetry(sample_id: int, db: Session = Depends(get_db)):
    ok = telemetry_service.delete_sample(db, sample_id)
    if not ok:
        raise HTTPException(status_code=404, detail="Telemetry sample not found")
    return None
```

main.py

```
from fastapi import FastAPI
from db import Base, engine
from routers import telemetry as telemetry_router

Base.metadata.create_all(bind=engine)

app = FastAPI(title="F1 Telemetry CRUD API")

app.include_router(telemetry_router.router)
```

Hit `http://127.0.0.1:8000/docs` and you'll see the same CRUD behaviour as before, now split across main/routers/schemas/services.[44][41][42]

Do you want to rename this stack to a more innovative model next (e.g. `LapSummary` instead of `TelemetrySample`), or first add a bulk import route in the router/service layer?

⁂

from db import Base, engine

ModuleNotFoundError: No module named 'db'

That error means Python can't find your `db.py` module from where you're running the app, usually because of package/layout issues, not the code itself.[45][46]

How to fix it

1. Check your files are in the same package

Simplest setup:

- project_root/
  - main.py
  - db.py
  - models.py
  - routers/
    - **init**.py

        - telemetry.py
    - schemas/
        - **init**.py
        - telemetry.py
    - services/
        - **init**.py
        - telemetry.py

`db.py` must sit next to `main.py` (same directory). Then in `main.py`, `models.py`, `routers/telemetry.py` you can do:

```
from db import Base, engine, get_db
```

If `db.py` is actually inside a package (e.g. `app/db.py`), you must import with the package name:

```
from app.db import Base, engine, get_db
```

2. Run uvicorn from the project root

From the folder that contains `main.py` and `db.py`:

```
uvicorn main:app --reload
```

If you're inside `routers/` when you run uvicorn, Python's import path won't include the parent directory, so `from db import ...` will fail.[47][45]

3. Use a package-style layout (optional but clean)

If you have:

- app/
    - **init**.py
    - main.py
    - db.py
    - models.py
    - routers/

    - schemas/
    - services/

Then:

- Imports inside the package use `from app.db import Base, engine`.
- You run uvicorn from the parent directory:

```
uvicorn app.main:app --reload
```

4. Quick diagnostic

In a Python shell from your project root:

```
import db
```

If that raises `ModuleNotFoundError`, your working directory or file locations are wrong.

Could you paste your current folder structure (just names/paths) so I can give you the exact import paths and uvicorn command that will work for your setup?

⁂

ModuleNotFoundError: No module named 'app'

That error means your package name in the import/uvicorn command doesn't match your actual folder structure.[48][49]

Check your layout

From your project root, run:

```
ls
```

If you see:

- main.py

- db.py
- routers/
- schemas/
- services/

and no `app/` folder, then you should NOT be using `from app.db import ...` or `uvicorn app.main:app`. Instead use plain names:

- Imports:

```
# in main.py, models.py, routers/*
from db import Base, engine, get_db
```

- Run:

```
uvicorn main:app --reload
```

If you actually have:

- app/
  - **init**.py
  - main.py
  - db.py
  - routers/
  - schemas/
  - services/

then:

- Imports inside that package:

```
from app.db import Base, engine, get_db
from app.routers import telemetry
```

- Run from the parent directory:

```
uvicorn app.main:app --reload
```

One simple working option

For now, put everything in a flat project (no `app` folder):

- project_root/
  - main.py
  - db.py
  - models.py
  - routers/
    - **init**.py
    - telemetry.py

Then:

`db.py`:

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base

SQLALCHEMY_DATABASE_URL = "sqlite:///./telemetry.db"

engine = create_engine(
    SQLALCHEMY_DATABASE_URL, connect_args={"check_same_thread": False}
)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

`main.py`:

```
from fastapi import FastAPI
from db import Base, engine
from routers import telemetry as telemetry_router

Base.metadata.create_all(bind=engine)
```

```
app = FastAPI()
app.include_router(telemetry_router.router)
```

Run from the same folder:

```
uvicorn main:app --reload
```

What do you see when you list the files in your project root (just the names)? That will let me tell you the exact import/uvicorn line to use.

**

File "/Users/Sam/anaconda3/lib/python3.11/multiprocessing/process.py", line 314, in _bootstrap

```
self.run()
```

File "/Users/Sam/anaconda3/lib/python3.11/multiprocessing/process.py", line 108, in run
self._target(*self._args, **self._kwargs)
File "/Users/Sam/anaconda3/lib/python3.11/site-packages/uvicorn/_subprocess.py", line 80, in subprocess_started
target(sockets=sockets)
File "/Users/Sam/anaconda3/lib/python3.11/site-packages/uvicorn/server.py", line 67, in run
return asyncio_run(self.serve(sockets=sockets), loop_factory=self.config.get_loop_factory())
^^^

File "/Users/Sam/anaconda3/lib/python3.11/site-packages/uvicorn/server.py", line 71, in serve
await self._serve(sockets)
File "/Users/Sam/anaconda3/lib/python3.11/site-packages/uvicorn/server.py", line 78, in _serve
config.load()
File "/Users/Sam/anaconda3/lib/python3.11/site-packages/uvicorn/config.py", line 439, in load
self.loaded_app = import_from_string(self.app)
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/Users/Sam/anaconda3/lib/python3.11/site-packages/uvicorn/importer.py", line 22, in import_from_string
raise exc from None
File "/Users/Sam/anaconda3/lib/python3.11/site-packages/uvicorn/importer.py", line 19, in import_from_string
module = importlib.import_module(module_str)
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/Users/Sam/anaconda3/lib/python3.11/importlib/**init**.py", line 126, in import_module
return _bootstrap._gcd_import(name[level:], package, level)
^^^
File "<frozen importlib._bootstrap>", line 1204, in _gcd_import
File "<frozen importlib._bootstrap>", line 1176, in _find_and_load
File "<frozen importlib._bootstrap>", line 1126, in _find_and_load_unlocked
File "<frozen importlib._bootstrap>", line 241, in _call_with_frames_removed
File "<frozen importlib._bootstrap>", line 1204, in _gcd_import
File "<frozen importlib._bootstrap>", line 1176, in _find_and_load
File "<frozen importlib._bootstrap>", line 1140, in _find_and_load_unlocked
ModuleNotFoundError: No module named 'app'

The error happens because you're running `uvicorn app.main:app` but there's no `app` module (folder) in your project.[50][51]

Quick fix: use flat structure, no `app` folder

1. Your project root should look like this:

```
project_root/
├── main.py
├── db.py
├── models.py
├── routers/
│   ├── __init__.py
```

```
│   └── telemetry.py
├── schemas/
│   ├── __init__.py
│   └── telemetry.py
└── services/
    ├── __init__.py
    └── telemetry.py
```

2. Imports use plain names (no `app.` prefix):

In `main.py`:

```
from fastapi import FastAPI
from db import Base, engine
from routers import telemetry as telemetry_router  # ← no "app."

Base.metadata.create_all(bind=engine)

app = FastAPI(title="F1 Telemetry API")
app.include_router(telemetry_router.router)
```

In `models.py`:

```
from sqlalchemy import Column, Integer, String, Float, Boolean
from db import Base  # ← no "app."
```

In `routers/telemetry.py`:

```
from db import get_db  # ← no "app."
from services.telemetry import *  # ← no "app."
```

3. Run from project root (where main.py lives):

```
cd /path/to/project_root
uvicorn main:app --reload
```

Not:

```
uvicorn app.main:app  # ❌ This assumes an "app/" folder
```

Test it works

After running `uvicorn main:app --reload`:

- Visit `http://127.0.0.1:8000/docs`
- Try POST to `/api/v1/telemetry` with sample JSON
- All CRUD endpoints should work

If you want an `app/` package later

Only then use `uvicorn app.main:app --reload` and prefix imports with `app.`:

```
project_root/
└── app/
    ├── __init__.py
    ├── main.py
    ├── db.py
    └── ...
```

Run from `project_root/`:

```
uvicorn app.main:app --reload
```

Paste your current `ls` output (file/folder names in project root) and I'll give you the exact command + imports that match.

This will get your CRUD running in 2 minutes.

⁂

from db import Base, engine

ImportError: cannot import name 'Base' from 'db' (/Users/Sam/Documents/GitHub/Web_Ser_API/app/db.py)

The "partially initialized module" error means there's a **circular import** or your `db.py` file has a problem where `Base` isn't defined when `models.py` tries to import it.[60]

Most likely cause: circular imports or `db.py` syntax error

1. Check your `db.py` exactly matches this (copy-paste):

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base

SQLALCHEMY_DATABASE_URL = "sqlite:///./telemetry.db"

engine = create_engine(
    SQLALCHEMY_DATABASE_URL, connect_args={"check_same_thread": False}
)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

Base = declarative_base()  # ← This line MUST be here

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

2. Check your `models.py` exactly:

```
from sqlalchemy import Column, Integer, String, Float, Boolean
from db import Base  # ← Only this import

class TelemetrySample(Base):
    __tablename__ = "telemetry_samples"

    id = Column(Integer, primary_key=True, index=True)
    session_id = Column(String, index=True, nullable=False)
    driver_code = Column(String, index=True, nullable=False)
    lap_number = Column(Integer, nullable=False)
    sample_time_ms = Column(Float, nullable=False)
    speed_kph = Column(Float, nullable=False)
    rpm = Column(Integer, nullable=False)
    gear = Column(Integer, nullable=False)
    throttle_pct = Column(Float, nullable=False)
    brake = Column(Boolean, nullable=False)
    drs_active = Column(Boolean, nullable=False)
```

3. Test in Python shell (from project root):

```
python
>>> from db import Base
>>> from models import TelemetrySample
>>> print(Base)
<class 'sqlalchemy.orm.decl_api.DeclarativeBase'>
>>> exit()
```

If that fails, `db.py` has a syntax error or missing import.

4. Common circular import traps to avoid:

❌ **Don't** do this in `models.py`:

```
from db import Base, engine  # Don't import engine here
```

❌ **Don't** do this in `db.py`:

```
from models import *  # Never
```

Quick diagnostic commands

From your project root (`/Users/Sam/Documents/GitHub/Web_Ser_API/app/`?):

```
python -c "from db import Base; print('db OK')"
python -c "from models import TelemetrySample; print('models OK')"
```

Paste the output of those two commands, and if either fails I'll tell you exactly what's wrong in your file.

Nuclear option: minimal test

Create a fresh `test_main.py` in your project root:

```
from fastapi import FastAPI
from db import Base, engine
print("Imports work!")  # If you see this, imports are fine


python test_main.py
```

If that fails, the problem is in `db.py` itself (syntax error, missing sqlalchemy, etc.).

What does `python -c "from db import Base"` print?

⁂

You can keep the data “real” and still satisfy CRUD by treating your API as a telemetry curation and analytics layer on top of existing sources like OpenF1, not as a fake data generator.openf1+1

Big idea

Instead of “creating” telemetry values, your API’s job is to:

Ingest authoritative telemetry from upstream sources (OpenF1, FastF1, your CSV uploads) into your own database in a controlled way.openf1+1

Let users curate, annotate, slice, and analyse that telemetry via full CRUD on your own domain objects (sessions, driver runs, laps, annotations, favourites, derived metrics), while the raw numbers remain grounded in real data.[github]

So “C” is not “invent numbers”, it is “create a record in your system that references real telemetry”.

Innovative data model options

Pick a higher-level entity than raw samples for CRUD, and keep samples read-only:

session_dataset: “All telemetry for VER in AUS 2025 race” loaded from OpenF1 and stored once.

lap_summary: one row per lap with averages, extremes, and gaps, computed from raw telemetry.[youtube][dev]

comparison_run: “HAM vs VER lap 12” storing config + cached comparison metrics (gap vs distance, sector losses, etc.).dev+1

annotation: user-added comments/labels on laps, corners, or intervals (“lockup”, “great exit”, etc.).[reddit]

Each of these can have full CRUD, even though the underlying telemetry stream is immutable.

How CRUD can look without inventing telemetry

Create

You can define “Create” as creating curated telemetry artefacts, not raw samples:

POST /api/v1/sessions/import

Body: { source: "openf1", session_key: "2023, Bahrain, Race" }.

Behaviour: call OpenF1 /car_data, /laps, etc., pull real data, store it as your session_dataset row plus its associated samples/laps in your DB.github+2

This is creating a new representation of real data inside your system, not fabricating values.

POST /api/v1/lap_summaries

Body: { session_id, driver_code, lap_number }.

Behaviour: compute summary metrics from stored samples (avg speed, max speed, time loss in sectors) and persist them as a new lap_summary row.[youtube][dev]

POST /api/v1/comparisons

Body: { session_id, driver_a, driver_b, lap_number }.

Behaviour: run a gap-vs-distance analysis and store the result as a comparison_run resource with metadata and maybe a pre-computed trace for plotting later.dev+1

POST /api/v1/annotations

Body: { session_id, driver_code, lap_number, start_distance_m, end_distance_m, note: "Tyre warmup issues" }.

Behaviour: attach user commentary to real telemetry intervals.[reddit]

In your report you can say: “Create operations never fabricate telemetry; they create curated resources (imports, summaries, comparisons, annotations) derived from or linked to real F1 data.”

Read

Read becomes very rich and clearly “real”:

GET /api/v1/sessions/{id} → shows which real session you imported, its drivers and laps.openf1+1

GET /api/v1/lap_summaries?session_id=…&driver_code=… → table of real lap metrics.[youtube][dev]

GET /api/v1/comparisons/{id} → returns the stored comparison_run for visualisation.[reddit]

GET /api/v1/annotations?session_id=… → overlays user notes on real telemetry.[reddit]

Update

You don’t update raw telemetry, you update your curation layer:

PUT/PATCH /api/v1/sessions/{id}

Change local labels (e.g. “Bahrain 2025 Race – Verstappen data only”) or visibility.

PUT/PATCH /api/v1/annotations/{id}

Fix or expand a note about an oversteer moment or tyre phase.

PUT/PATCH /api/v1/comparisons/{id}

Adjust options (alignment by time vs by distance, smoothing settings) and re-compute in the background.dev+1

Delete

Delete also operates at the curated level:

DELETE /api/v1/sessions/{id} → drop that imported session (and related summaries/comparisons)

from your DB.
DELETE /api/v1/annotations/{id} → remove a user note.
DELETE /api/v1/comparisons/{id} → remove a stored comparison you no longer need.
Again, raw F1 data is authoritative; you're only removing your copy or your added layers.
Two concrete design patterns
You could frame the assignment using one of these patterns:
Telemetry import & summary API
Core model for CRUD: lap_summary.
Create: compute & store new lap summaries for a driver's laps after importing real telemetry.
Read: filter lap summaries for sessions/drivers; get one by id.
Update: correct labels/metadata on a summary (e.g. "push lap", "cooling lap").
Delete: remove summaries you don't want to keep.
Driver comparison API
Core model for CRUD: comparison_run.
Create: define a comparison between two real laps and compute gap-vs-distance + key metrics, store it.
Read: list and fetch comparisons, maybe with filters by drivers or track.[reddit]
Update: tweak parameters (window size, alignment method) and re-generate.
Delete: remove old comparisons.
Both are clearly grounded in real telemetry and feel more "innovative" than just CRUD over a raw_sample table, while still satisfying the coursework requirement that you implement CRUD on at least one data model linked to a database.[perplexity]
Which direction feels more exciting for you: a lap_summary/analytics-first API, a comparison_run/driver-vs-driver API, or an annotation-layer API where users mark up interesting telemetry moments? i know need to convert my current application wiht basic cruds for f1 telemetry to this

You don't need to throw away your existing basic CRUD; you can refactor it so the **CRUD model is "curated telemetry" (lap_summary / comparison / session_dataset)** instead of raw samples, and keep your current structure (routers, schemas, services, db) almost unchanged.[70][71]

Pick your new core model

Given what you wrote, the cleanest upgrade is usually **lap_summary** (per-lap analytics) or **comparison_run** (driver-vs-driver). I'll show lap_summary; you can mirror the pattern for comparison later.[72][70]

A `LapSummary` row might look like:

- id
- session_key (OpenF1 session_key)
- driver_number
- lap_number
- lap_duration
- max_speed_kph
- avg_speed_kph
- i1_speed, i2_speed, st_speed (optionally)
- label (e.g. “push lap”, “cool lap”)

These fields map nicely to OpenF1’s `laps` and speed attributes.[70][72]

Your existing TelemetrySample model then becomes **read-only internal detail**, and the full CRUD you demonstrate for the assignment is on `LapSummary`.

Step 1: change the model (models.py)

Replace your `TelemetrySample` with a `LapSummary` model:

```
# models.py
from sqlalchemy import Column, Integer, String, Float
from db import Base

class LapSummary(Base):
    __tablename__ = "lap_summaries"

    id = Column(Integer, primary_key=True, index=True)
    session_key = Column(Integer, index=True, nullable=False)  # OpenF1 session_key
    driver_number = Column(Integer, index=True, nullable=False)
    lap_number = Column(Integer, nullable=False)

    lap_duration = Column(Float, nullable=False)      # seconds
    max_speed_kph = Column(Float, nullable=True)
    avg_speed_kph = Column(Float, nullable=True)
    i1_speed = Column(Float, nullable=True)
    i2_speed = Column(Float, nullable=True)
    st_speed = Column(Float, nullable=True)
```

```
    label = Column(String, nullable=True)  # "push", "cool", "in-lap", etc.
```

Then in `main.py` (or wherever you call `Base.metadata.create_all`), this table will be created automatically.[73][74]

Step 2: update schemas to LapSummary

In `schemas/lap_summary.py` (or reuse your telemetry schema file but rename classes):

```
# schemas/lap_summary.py
from typing import Optional
from pydantic import BaseModel, Field

class LapSummaryBase(BaseModel):
    session_key: int = Field(..., example=9159)
    driver_number: int = Field(..., example=1)
    lap_number: int = Field(..., ge=1, example=12)
    lap_duration: float = Field(..., gt=0, example=93.456)
    max_speed_kph: Optional[float] = Field(None, ge=0, example=320.5)
    avg_speed_kph: Optional[float] = Field(None, ge=0, example=210.3)
    i1_speed: Optional[float] = Field(None, ge=0, example=295.0)
    i2_speed: Optional[float] = Field(None, ge=0, example=305.0)
    st_speed: Optional[float] = Field(None, ge=0, example=318.0)
    label: Optional[str] = Field(None, example="push")

class LapSummaryCreate(LapSummaryBase):
    pass

class LapSummaryUpdate(BaseModel):
    # allow partial updates
    lap_duration: Optional[float] = Field(None, gt=0)
    max_speed_kph: Optional[float] = Field(None, ge=0)
    avg_speed_kph: Optional[float] = Field(None, ge=0)
    i1_speed: Optional[float] = Field(None, ge=0)
    i2_speed: Optional[float] = Field(None, ge=0)
    st_speed: Optional[float] = Field(None, ge=0)
    label: Optional[str] = None

class LapSummaryRead(LapSummaryBase):
    id: int

    class Config:
        orm_mode = True
```

This keeps your previous pattern but for a **curated lap**.[74][73]

Step 3: adapt services to LapSummary CRUD

In `services/lap_summary.py`:

```
# services/lap_summary.py
from typing import List, Optional
from sqlalchemy.orm import Session
from models import LapSummary
from schemas.lap_summary import (
    LapSummaryCreate,
    LapSummaryUpdate,
)

def create_lap_summary(db: Session, lap_in: LapSummaryCreate) -> LapSummary:
    lap = LapSummary(**lap_in.dict())
    db.add(lap)
    db.commit()
    db.refresh(lap)
    return lap

def list_lap_summaries(
    db: Session,
    session_key: Optional[int] = None,
    driver_number: Optional[int] = None,
) -> List[LapSummary]:
    q = db.query(LapSummary)
    if session_key is not None:
        q = q.filter(LapSummary.session_key == session_key)
    if driver_number is not None:
        q = q.filter(LapSummary.driver_number == driver_number)
    return q.all()

def get_lap_summary(db: Session, lap_id: int) -> Optional[LapSummary]:
    return db.query(LapSummary).get(lap_id)

def update_lap_summary(
    db: Session, lap_id: int, lap_update: LapSummaryUpdate
) -> Optional[LapSummary]:
    lap = db.query(LapSummary).get(lap_id)
    if not lap:
        return None
    data = lap_update.dict(exclude_unset=True)
```

```
    for field, value in data.items():
        setattr(lap, field, value)
    db.commit()
    db.refresh(lap)
    return lap

def delete_lap_summary(db: Session, lap_id: int) -> bool:
    lap = db.query(LapSummary).get(lap_id)
    if not lap:
        return False
    db.delete(lap)
    db.commit()
    return True
```

Business logic stays the same; only the model and schema names change.[73][74]

Step 4: switch router to LapSummary

In `routers/lap_summary.py`:

```
# routers/lap_summary.py
from typing import List, Optional
from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.orm import Session

from db import get_db
from schemas.lap_summary import (
    LapSummaryCreate,
    LapSummaryUpdate,
    LapSummaryRead,
)
from services import lap_summary as lap_service

router = APIRouter(prefix="/api/v1/lap_summaries", tags=["lap_summaries"])

@router.post(
    "",
    response_model=LapSummaryRead,
    status_code=status.HTTP_201_CREATED,
)
def create_lap_summary(
    lap_in: LapSummaryCreate,
    db: Session = Depends(get_db),
):
```

```
    return lap_service.create_lap_summary(db, lap_in)

@router.get("", response_model=List[LapSummaryRead])
def list_lap_summaries(
    session_key: Optional[int] = None,
    driver_number: Optional[int] = None,
    db: Session = Depends(get_db),
):
    return lap_service.list_lap_summaries(db, session_key, driver_number)

@router.get("/{lap_id}", response_model=LapSummaryRead)
def get_lap_summary(lap_id: int, db: Session = Depends(get_db)):
    lap = lap_service.get_lap_summary(db, lap_id)
    if not lap:
        raise HTTPException(status_code=404, detail="Lap summary not found")
    return lap

@router.patch("/{lap_id}", response_model=LapSummaryRead)
def update_lap_summary(
    lap_id: int,
    lap_update: LapSummaryUpdate,
    db: Session = Depends(get_db),
):
    lap = lap_service.update_lap_summary(db, lap_id, lap_update)
    if not lap:
        raise HTTPException(status_code=404, detail="Lap summary not found")
    return lap

@router.delete("/{lap_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_lap_summary(lap_id: int, db: Session = Depends(get_db)):
    ok = lap_service.delete_lap_summary(db, lap_id)
    if not ok:
        raise HTTPException(status_code=404, detail="Lap summary not found")
    return None
```

And in `main.py`:

```
from fastapi import FastAPI
from db import Base, engine
from routers import lap_summary as lap_router

Base.metadata.create_all(bind=engine)

app = FastAPI(title="F1 Lap Summary API")
```

```
app.include_router(lap_router.router)
```

Now your **CRUD is for LapSummary**, which is exactly the "curated telemetry artefact" idea.[72][70]

Step 5: optionally add an OpenF1 import endpoint

Once CRUD works locally, you can add a single "innovative" endpoint that pulls real data from OpenF1 and populates LapSummary rows.

In the same router:

```
import httpx
from fastapi import BackgroundTasks

@router.post("/import_from_openf1", status_code=status.HTTP_202_ACCEPTED)
async def import_lap_summaries(
    session_key: int,
    driver_number: int,
    background_tasks: BackgroundTasks,
    db: Session = Depends(get_db),
):
    # In a real app, put this in a service + background task
    async with httpx.AsyncClient() as client:
        resp = await client.get(
            "https://api.openf1.org/v1/laps",
            params={"session_key": session_key, "driver_number": driver_number},
        )
        resp.raise_for_status()
        laps = resp.json()

    for lap in laps:
        lap_in = LapSummaryCreate(
            session_key=session_key,
            driver_number=driver_number,
            lap_number=lap["lap_number"],
            lap_duration=lap["lap_duration"],
            max_speed_kph=lap.get("st_speed"),
            avg_speed_kph=None,   # or compute from car_data later
            i1_speed=lap.get("i1_speed"),
            i2_speed=lap.get("i2_speed"),
            st_speed=lap.get("st_speed"),
            label=None,
        )
        lap_service.create_lap_summary(db, lap_in)
```

```
    return {"imported": len(laps)}
```

That uses OpenF1's `laps` endpoint attributes (`lap_number`, `lap_duration`, `i1_speed`, `i2_speed`, `st_speed`) to populate your curated LapSummary table.[71][70][72]

CRUD coverage summary (for your report)

Layer	Entity	C	R	U	D
Curated layer	LapSummary	POST /api/v1/lap_summaries	GET list / GET by id	PATCH /api/v1/lap_summaries/{lap_id}	DELETE /api/v1/lap_summaries/{id}
Import utility	LapSummary	POST /api/v1/lap_summaries/import*	(uses LapSummary reads later)	n/a	n/a
Raw telemetry	OpenF1	n/a (read-only external)	fetched via OpenF1 `laps`, `car_data` endpoints	n/a	n/a

*Import endpoint derives LapSummary rows from real OpenF1 data.[71][70][72]

Would you like the same refactor pattern shown for a `ComparisonRun` model next, or do you want to first make sure your LapSummary CRUD compiles and shows up correctly in `/docs`?

⁂

lets break this down more i need to first create a requrest to impote data and store it

You first need one **"import" endpoint** that (1) receives a request, (2) calls OpenF1 (or another source), and (3) stores curated rows (e.g. LapSummary) in your DB.

Below is a minimal, concrete version of that flow.

1. Decide the import request shape

For lap summaries, a simple body is:

```
{
  "session_key": 9165,
  "driver_number": 1
}
```

This says: "Import all laps for driver 1 in session 9165 into my lap_summaries table."[79][80]

Define a schema:

```
# schemas/import_request.py
from pydantic import BaseModel, Field

class LapImportRequest(BaseModel):
    session_key: int = Field(..., example=9165)
    driver_number: int = Field(..., example=1)
```

2. Service function: fetch external data and store

In `services/lap_summary.py` add an import function that:

- Calls OpenF1 `/laps`.
- Maps each lap to a LapSummaryCreate.
- Uses your existing create service to save rows.[81][79]

```
# services/lap_summary.py
from typing import List
import httpx
from sqlalchemy.orm import Session
from models import LapSummary
from schemas.lap_summary import LapSummaryCreate
from schemas.import_request import LapImportRequest

OPENF1_BASE_URL = "https://api.openf1.org/v1"

async def import_lap_summaries_from_openf1(
    db: Session,
```

```
    req: LapImportRequest,
) -> int:
    # 1) call OpenF1 laps endpoint
    async with httpx.AsyncClient(base_url=OPENF1_BASE_URL) as client:
        resp = await client.get(
            "/laps",
            params={
                "session_key": req.session_key,
                "driver_number": req.driver_number,
            },
        )
        resp.raise_for_status()
        laps: List[dict] = resp.json()

    # 2) convert each lap to LapSummaryCreate and store
    count = 0
    for lap in laps:
        lap_in = LapSummaryCreate(
            session_key=req.session_key,
            driver_number=req.driver_number,
            lap_number=lap["lap_number"],
            lap_duration=lap["lap_duration"],          # OpenF1 field[web:24]
            max_speed_kph=lap.get("st_speed"),         # use speed trap as
proxy[web:24]
            avg_speed_kph=None,                        # keep None or compute later
            i1_speed=lap.get("i1_speed"),
            i2_speed=lap.get("i2_speed"),
            st_speed=lap.get("st_speed"),
            label=None,
        )
        db_lap = LapSummary(**lap_in.dict())
        db.add(db_lap)
        count += 1

    db.commit()
    return count
```

This is your **core import logic**: it keeps the data "real" by pulling from OpenF1 and only creating curated rows based on that.[80][81][79]

3. Router endpoint: expose the import request

In `routers/lap_summary.py` add:

```
from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.orm import Session
from db import get_db
from schemas.import_request import LapImportRequest
from services import lap_summary as lap_service

router = APIRouter(prefix="/api/v1/lap_summaries", tags=["lap_summaries"])

@router.post("/import_from_openf1", status_code=status.HTTP_202_ACCEPTED)
async def import_from_openf1(
    req: LapImportRequest,
    db: Session = Depends(get_db),
):
    try:
        count = await lap_service.import_lap_summaries_from_openf1(db, req)
    except httpx.HTTPError as e:
        raise HTTPException(
            status_code=502,
            detail=f"Failed to fetch from OpenF1: {e}",
        )
    return {"imported": count}
```

Now the workflow is:

1. Client sends POST `/api/v1/lap_summaries/import_from_openf1` with `{session_key, driver_number}`.
2. Your API calls OpenF1 `/laps`.
3. For each lap, you create a `LapSummary` row in your DB.
4. You then use your standard CRUD to Read/Update/Delete those LapSummary rows.

4. How this fits CRUD

- **Create:**
  - POST `/api/v1/lap_summaries` → manual creation of lap summaries (pure CRUD).
  - POST `/api/v1/lap_summaries/import_from_openf1` → bulk creation of lap summaries from real telemetry.[81][79]
- **Read:**
  - GET `/api/v1/lap_summaries?session_key=...&driver_number=...`

    - GET `/api/v1/lap_summaries/{id}`
- **Update:**
    - PATCH `/api/v1/lap_summaries/{id}` to adjust label/metrics.
- **Delete:**
    - DELETE `/api/v1/lap_summaries/{id}`

These CRUD ops are on your curated LapSummary model; the import endpoint is the innovative part that makes it clearly grounded in OpenF1 data.[79][80][81]

Would you like to see the exact minimal set of LapSummary CRUD endpoints again, but now including this import route, so you can paste them into your router without guessing?

⁂

ModuleNotFoundError: No module named 'httpx'

You're missing the `httpx` dependency in your current environment; install it, then restart your app.[89]

Install httpx

From your project's virtualenv (or base, if you're using Anaconda):

```
pip install httpx
```

If you use conda:

```
conda install -c conda-forge httpx
```

Or explicitly with Python 3:

```
python -m pip install httpx
```

Then stop uvicorn and start it again:

```
uvicorn main:app --reload
# or, if you're using an app package:
uvicorn app.main:app --reload
```

Now `import httpx` in your service file and the `/import_from_openf1` endpoint should load correctly.[90][89]

Do you want the import endpoint rewritten to use the standard `requests` library instead (which you may already have installed), or are you happy to stick with `httpx`?

⁂

its still not working

If `httpx` is installed but still "not working", it's almost always one of:

- Installed into a different environment than the one running uvicorn.
- Import path typo (e.g. `import Httpx` instead of `import httpx`).
- IDE using a different interpreter.

Here's how to pin it down quickly.

1. Confirm installation in the same env

Run these **from the same terminal** where you run uvicorn:

```
python -c "import sys; print(sys.executable)"
python -c "import httpx; print(httpx.__version__)"
```

- If the second command fails with `ModuleNotFoundError`, `httpx` is not installed in this Python.
- Install it for that interpreter:

```
python -m pip install httpx
```

Then run again:

```
python -c "import httpx; print(httpx.__version__)"
```

Only when that works, start uvicorn from the same terminal:

```
uvicorn main:app --reload
# or uvicorn app.main:app --reload if you have an app/ package
```

2. Check your import line

In your service file you must have exactly:

```
import httpx
```

Not:

- `from httpx import AsyncClient` (fine but then match usage).
- `import Httpx` (capital H).
- `from . import httpx` (wrong).

If you want to avoid httpx entirely, you can switch to `requests` instead.

3. Option: use requests instead of httpx

Replace the async import function with a sync one using `requests`:

```
# services/lap_summary.py
import requests

def import_lap_summaries_from_openf1(db: Session, req: LapImportRequest) -> int:
    resp = requests.get(
        "https://api.openf1.org/v1/laps",
        params={
            "session_key": req.session_key,
            "driver_number": req.driver_number,
        },
        timeout=10,
    )
    resp.raise_for_status()
    laps = resp.json()

    count = 0
    for lap in laps:
        lap_in = LapSummaryCreate(
            session_key=req.session_key,
            driver_number=req.driver_number,
            lap_number=lap["lap_number"],
```

```
            lap_duration=lap["lap_duration"],
            max_speed_kph=lap.get("st_speed"),
            avg_speed_kph=None,
            i1_speed=lap.get("i1_speed"),
            i2_speed=lap.get("i2_speed"),
            st_speed=lap.get("st_speed"),
            label=None,
        )
        db_lap = LapSummary(**lap_in.dict())
        db.add(db_lap)
        count += 1

    db.commit()
    return count
```

Router becomes sync as well:

```
@router.post("/import_from_openf1", status_code=status.HTTP_202_ACCEPTED)
def import_from_openf1(
    req: LapImportRequest,
    db: Session = Depends(get_db),
):
    try:
        count = lap_service.import_lap_summaries_from_openf1(db, req)
    except requests.RequestException as e:
        raise HTTPException(status_code=502, detail=f"Failed to fetch from OpenF1:
{e}")
    return {"imported": count}
```

Install requests if needed:

```
python -m pip install requests
```

To help precisely: what do you see **now** in the terminal when you start uvicorn—full error message including the line that mentions `httpx` or the module name? If you paste just that traceback, I can target the exact fix.

from typing import List, Optional

```
from fastapi import APIRouter, Depends, HTTPException, status, BackgroundTasks
from sqlalchemy.orm import Session
import httpx

from db import get_db
from schemas.lap_summary import (
LapSummaryCreate,
LapSummaryUpdate,
LapSummaryRead,
)
from services import lap_summary as lap_service

router = APIRouter(prefix="/api/v1/lap_summaries", tags=["lap_summaries"])

@router.post(
"",
response_model=LapSummaryRead,
status_code=status.HTTP_201_CREATED,
)
def create_lap_summary(
lap_in: LapSummaryCreate,
db: Session = Depends(get_db),
):
return lap_service.create_lap_summary(db, lap_in)

@router.post("/import_from_openf1", status_code=status.HTTP_202_ACCEPTED)
async def import_lap_summaries(
session_key: int,
driver_number: int,
background_tasks: BackgroundTasks,
db: Session = Depends(get_db),
):
# In a real app, put this in a service + background task
async with httpx.AsyncClient() as client:
resp = await client.get(
"https://api.openf1.org/v1/laps",
params={"session_key": session_key, "driver_number": driver_number},
)
```

resp.raise_for_status()

laps = resp.json()

```
for lap in laps:
    lap_in = LapSummaryCreate(
        session_key=session_key,
        driver_number=driver_number,
        lap_number=lap["lap_number"],
        lap_duration=lap["lap_duration"],
        max_speed_kph=lap.get("st_speed"),
        avg_speed_kph=None,   # or compute from car_data later
        i1_speed=lap.get("i1_speed"),
        i2_speed=lap.get("i2_speed"),
        st_speed=lap.get("st_speed"),
        label=None,
    )
    lap_service.create_lap_summary(db, lap_in)


return {"imported": len(laps)}
```

@router.get("", response_model=List[LapSummaryRead])

def list_lap_summaries(

session_key: Optional[int] = None,

driver_number: Optional[int] = None,

db: Session = Depends(get_db),

):

return lap_service.list_lap_summaries(db, session_key, driver_number)

@router.get("/{lap_id}", response_model=LapSummaryRead)

def get_lap_summary(lap_id: int, db: Session = Depends(get_db)):

lap = lap_service.get_lap_summary(db, lap_id)

if not lap:

raise HTTPException(status_code=404, detail="Lap summary not found")

return lap

@router.patch("/{lap_id}", response_model=LapSummaryRead)

def update_lap_summary(

lap_id: int,

lap_update: LapSummaryUpdate,

db: Session = Depends(get_db),

):
lap = lap_service.update_lap_summary(db, lap_id, lap_update)
if not lap:
raise HTTPException(status_code=404, detail="Lap summary not found")
return lap

@router.delete("/{lap_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_lap_summary(lap_id: int, db: Session = Depends(get_db)):
ok = lap_service.delete_lap_summary(db, lap_id)
if not ok:
raise HTTPException(status_code=404, detail="Lap summary not found")
return None

Your router code is fine; the issue is still environment/import, not the logic. The quickest path is to avoid httpx for now and make this endpoint fully sync with `requests`, which you can get working in seconds.

1. Replace httpx with requests in this file

At the top:

```
from typing import List, Optional
from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.orm import Session
import requests  # ← use requests instead of httpx

from db import get_db
from schemas.lap_summary import (
    LapSummaryCreate,
    LapSummaryUpdate,
    LapSummaryRead,
)
from services import lap_summary as lap_service
```

2. Make the import endpoint sync

Replace your `import_lap_summaries` with:

```
@router.post("/import_from_openf1", status_code=status.HTTP_202_ACCEPTED)
def import_lap_summaries(
    session_key: int,
```

```
    driver_number: int,
    db: Session = Depends(get_db),
):
    try:
        resp = requests.get(
            "https://api.openf1.org/v1/laps",
            params={"session_key": session_key, "driver_number": driver_number},
            timeout=10,
        )
        resp.raise_for_status()
    except requests.RequestException as e:
        raise HTTPException(status_code=502, detail=f"Failed to fetch from OpenF1:
{e}")

    laps = resp.json()

    for lap in laps:
        lap_in = LapSummaryCreate(
            session_key=session_key,
            driver_number=driver_number,
            lap_number=lap["lap_number"],
            lap_duration=lap["lap_duration"],
            max_speed_kph=lap.get("st_speed"),
            avg_speed_kph=None,
            i1_speed=lap.get("i1_speed"),
            i2_speed=lap.get("i2_speed"),
            st_speed=lap.get("st_speed"),
            label=None,
        )
        lap_service.create_lap_summaries(db, lap_in)  # or create_lap_summary if that's
your name

    return {"imported": len(laps)}
```

Adjust the service call to match your function name (e.g. `create_lap_summary`).

3. Install requests in the same env

From the same terminal where you run uvicorn:

```
python -m pip install requests
```

Then restart:

```
uvicorn main:app --reload
# or uvicorn app.main:app --reload if you have an app package
```

Now you should see `/api/v1/lap_summaries/import_from_openf1` in `/docs` and be able to POST with `session_key` and `driver_number` to import laps.

Once this is running, if you still want async/httpx we can switch back—but first, confirm this sync version works.

fix now when i run the import it pydantic_core._pydantic_core.ValidationError: 1 validation error for LapSummaryCreate

lap_duration
Input should be a valid number [type=float_type, input_value=None, input_type=NoneType]
For further information visit https://errors.pydantic.dev/2.12/v/float_type

`lap_duration` is coming back as `null` from OpenF1 for at least one lap, but your `LapSummaryCreate` schema requires a non-null float, so Pydantic rejects it.

1. Make lap_duration optional in the schema

In `schemas/lap_summary.py` change:

```
class LapSummaryBase(BaseModel):
    session_key: int
    driver_number: int
    lap_number: int = Field(..., ge=1)
    lap_duration: float = Field(..., gt=0)
```

to:

```
from typing import Optional

class LapSummaryBase(BaseModel):
    session_key: int
    driver_number: int
    lap_number: int = Field(..., ge=1)
```

```
    lap_duration: Optional[float] = Field(None, gt=0)
```

And in `LapSummaryCreate` just inherit:

```
class LapSummaryCreate(LapSummaryBase):
    pass
```

This lets you store laps where OpenF1 has no duration yet (e.g. in-laps, invalid laps).[91][92]

2. Map OpenF1 safely in the import loop

In your import endpoint, change:

```
lap_in = LapSummaryCreate(
    session_key=session_key,
    driver_number=driver_number,
    lap_number=lap["lap_number"],
    lap_duration=lap["lap_duration"],
    ...
)
```

to:

```
lap_in = LapSummaryCreate(
    session_key=session_key,
    driver_number=driver_number,
    lap_number=lap["lap_number"],
    lap_duration=lap.get("lap_duration"),  # may be None
    max_speed_kph=lap.get("st_speed"),
    avg_speed_kph=None,
    i1_speed=lap.get("i1_speed"),
    i2_speed=lap.get("i2_speed"),
    st_speed=lap.get("st_speed"),
    label=None,
)
```

If you prefer to **skip** laps with no duration instead of storing them, you can guard:

```
for lap in laps:
    if lap.get("lap_duration") is None:
        continue
```

```
    ...
```

Either approach will remove the ValidationError.[91]

Do you want to keep laps with missing durations (for completeness) or skip them entirely during import?

⁂

lap_service.create_lap_summaries(db, lap_in) this line is erroing what do i need todo

That line errors because your service function is almost certainly named `create_lap_summary`, not `create_lap_summaries`, so Python can't find it.

What to change

In `services/lap_summary.py` you likely have something like:

```
def create_lap_summary(db: Session, lap_in: LapSummaryCreate) -> LapSummary:
    ...
```

So in your router, change the call to match the function name:

```
for lap in laps:
    lap_in = LapSummaryCreate(
        session_key=session_key,
        driver_number=driver_number,
        lap_number=lap["lap_number"],
        lap_duration=lap.get("lap_duration"),
        max_speed_kph=lap.get("st_speed"),
        avg_speed_kph=None,
        i1_speed=lap.get("i1_speed"),
        i2_speed=lap.get("i2_speed"),
        st_speed=lap.get("st_speed"),
        label=None,
    )
    lap_service.create_lap_summary(db, lap_in)  # ← singular
```

If you prefer a bulk helper, you can define it in `services/lap_summary.py`:

```
def create_lap_summaries(db: Session, laps_in: List[LapSummaryCreate]) -> int:
    for lap_in in laps_in:
        db_lap = LapSummary(**lap_in.dict())
        db.add(db_lap)
    db.commit()
    return len(laps_in)
```

Then in the router you would **build a list and call it once**:

```
lap_inputs = []
for lap in laps:
    lap_inputs.append(
        LapSummaryCreate(
            session_key=session_key,
            driver_number=driver_number,
            lap_number=lap["lap_number"],
            lap_duration=lap.get("lap_duration"),
            max_speed_kph=lap.get("st_speed"),
            avg_speed_kph=None,
            i1_speed=lap.get("i1_speed"),
            i2_speed=lap.get("i2_speed"),
            st_speed=lap.get("st_speed"),
            label=None,
        )
    )

count = lap_service.create_lap_summaries(db, lap_inputs)
return {"imported": count}
```

But right now, the minimal fix is: make the function name in the router exactly match the one you defined in the service.

Can you paste your current `services/lap_summary.py` so I can confirm the function names line-by-line?

sqlalchemy.exc.IntegrityError: (sqlite3.IntegrityError) NOT NULL constraint failed: lap_summaries.lap_duration

[SQL: INSERT INTO lap_summaries (session_key, driver_number, lap_number, lap_duration, max_speed_kph, avg_speed_kph, i1_speed, i2_speed, st_speed, label) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)]
[parameters: (9161, 63, 1, None, None, None, 246.0, 254.0, None, None)]
(Background on this error at: https://sqlalche.me/e/14/gkpj)

The DB column still has `NOT NULL` on `lap_duration`, so SQLite rejects rows where OpenF1 sends `null` (None), even though your Pydantic schema now allows it.

You must update the **SQLAlchemy model** to make `lap_duration` nullable and recreate/migrate the table.

1. Fix the SQLAlchemy model

In `models.py`, change your `LapSummary` definition from something like:

```
lap_duration = Column(Float, nullable=False)
```

to:

```
lap_duration = Column(Float, nullable=True)
```

Make sure `nullable=True` (or omit it, default is True), not `nullable=False`.

2. Drop/recreate the table (quick dev reset)

Because you're on SQLite and in dev, easiest is to drop the table and let `create_all` recreate it with the new nullability.

From a Python shell in project root:

```
from db import engine, Base
from models import LapSummary
Base.metadata.drop_all(bind=engine, tables=[LapSummary.__table__])
Base.metadata.create_all(bind=engine, tables=[LapSummary.__table__])
```

Or, if you're happy to wipe everything:

```
from db import engine, Base
Base.metadata.drop_all(bind=engine)
```

```
Base.metadata.create_all(bind=engine)
```

Then restart uvicorn.

Now the column `lap_duration` accepts NULL and the INSERT with `None` will succeed.

3. Ensure your import still uses .get()

Double-check in your import loop:

```
lap_in = LapSummaryCreate(
    session_key=session_key,
    driver_number=driver_number,
    lap_number=lap["lap_number"],
    lap_duration=lap.get("lap_duration"),  # may be None
    max_speed_kph=lap.get("st_speed"),
    avg_speed_kph=None,
    i1_speed=lap.get("i1_speed"),
    i2_speed=lap.get("i2_speed"),
    st_speed=lap.get("st_speed"),
    label=None,
)
```

With Pydantic allowing `Optional[float]` and SQLAlchemy `nullable=True`, the IntegrityError will go away.

Do you want to keep wiping the table when you change the model, or shall we set up a simple Alembic migration so you can alter columns without losing data?

sqlalchemy.exc.IntegrityError: (sqlite3.IntegrityError) NOT NULL constraint failed: lap_summaries.lap_duration

[SQL: INSERT INTO lap_summaries (session_key, driver_number, lap_number, lap_duration, max_speed_kph, avg_speed_kph, i1_speed, i2_speed, st_speed, label) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)]
[parameters: (9161, 63, 1, None, None, None, 246.0, 254.0, None, None)]
(Background on this error at: https://sqlalche.me/e/14/gkpj)

SQLite is still enforcing `NOT NULL` on `lap_duration` in the actual table, even if you've changed the code. You need to (1) confirm the model, then (2) recreate or migrate the table so the DB column becomes nullable.

1. Confirm LapSummary model is nullable

In `models.py` it must look like:

```
class LapSummary(Base):
    __tablename__ = "lap_summaries"

    id = Column(Integer, primary_key=True, index=True)
    session_key = Column(Integer, index=True, nullable=False)
    driver_number = Column(Integer, index=True, nullable=False)
    lap_number = Column(Integer, nullable=False)

    lap_duration = Column(Float, nullable=True)  # ← THIS must be nullable=True
    max_speed_kph = Column(Float, nullable=True)
    avg_speed_kph = Column(Float, nullable=True)
    i1_speed = Column(Float, nullable=True)
    i2_speed = Column(Float, nullable=True)
    st_speed = Column(Float, nullable=True)
    label = Column(String, nullable=True)
```

If you still have `nullable=False` or omitted the argument but created the table earlier, the DB schema is stuck with `NOT NULL`.

2. Drop and recreate the table

Because this is SQLite in dev, easiest is to drop the `lap_summaries` table and recreate it.

From a Python shell in your project root:

```
python
>>> from db import engine, Base
>>> from models import LapSummary
>>> Base.metadata.drop_all(bind=engine, tables=[LapSummary.__table__])
>>> Base.metadata.create_all(bind=engine, tables=[LapSummary.__table__])
>>> exit()
```

Alternatively, if you're happy to blow away all tables:

```
>>> from db import engine, Base
>>> Base.metadata.drop_all(bind=engine)
>>> Base.metadata.create_all(bind=engine)
>>> exit()
```

Then restart uvicorn.

Now the DB column `lap_duration` will allow NULL, so rows with `lap_duration=None` from OpenF1 will insert without an IntegrityError.[93]

3. Keep the import using .get

Ensure your import code still does:

```
lap_duration=lap.get("lap_duration"),  # may be None
```

so laps with missing duration don't break Pydantic or SQLAlchemy.

After you do steps 1 and 2, hit the import endpoint again—it should succeed. If you're comfortable with this pattern, we can next add a small filter to skip laps with no duration or mark them with a label like `"no_time"`.

⁂

1. projects.f1_telemetry_api
2. https://www.perplexity.ai/search/3a1063b3-e72d-4f48-8683-e2cf502891e2
3. https://www.perplexity.ai/search/058ef7df-1cac-4a28-a4b9-0f45955c3ddb
4. https://www.perplexity.ai/search/7f8b6a71-6821-4016-b816-48fe256482be
5. https://www.perplexity.ai/search/7fb9ea21-7ce6-4830-9471-36474c07e336
6. https://www.perplexity.ai/search/2253aeff-a220-4555-9885-ac2723ea5b06
7. https://www.perplexity.ai/search/c6e4b3a9-05ae-4422-a53f-75b7de8be836
8. https://www.perplexity.ai/search/8dcf8dcb-0325-4377-903e-ad873ff0529a
9. https://docs.fastf1.dev/core.html
10. https://www.reddit.com/r/F1Technical/comments/1jpol3f/i_made_a_really_cool_website_to_visualize_the_raw/

11. https://www.reddit.com/r/Python/comments/xjf7w6/formula_1_api_my_project/

12. https://github.com/bordanattila/OpenF1_tutorial

13. https://www.linkedin.com/pulse/f1-project-querying-apis-using-python-arthur-ladwein-9lpec

14. https://blog.dataengineerthings.org/zero-lap-to-hero-building-a-formula-1-analytics-stack-with-python-dbt-and-postgres-f41409c79b6c

15. https://electraspeed.com/racecar-telemetry-systems

16. https://www.apica.io/blog/opentelemetry-best-practices-for-improving-your-monitoring-and-observability/

17. https://www.youtube.com/watch?v=VUgvhnQawbw

18. https://developers.hyprace.com

19. https://openf1.org

20. https://openf1.org/docs/

21. https://github.com/br-g/openf1

22. projects.f1_telemetry_api

23. https://www.youtube.com/watch?v=HSV2FhPtRVI

24. https://dev.to/ibrahimpima/how-im-building-a-racing-analysis-web-app-from-raw-telemetry-59g

25. https://www.reddit.com/r/F1Technical/comments/1peu34f/i_built_an_f1_telemetry_tool_that_goes_beyond/

26. https://github.com/br-g/openf1/blob/main/README.md

27. https://www.reddit.com/r/F1DataAnalysis/comments/16w84uz/openf1_an_api_for_realtime_f1_data/

28. https://mcpmarket.com/server/open-f1

29. https://desklib.com/study-documents/race-entry-system-design/

30. https://www.perplexity.ai/search/058ef7df-1cac-4a28-a4b9-0f45955c3ddb

31. https://dev.to/rock_win_c053fa5fb2399067/building-a-crud-application-with-fastapi-a-complete-guide-1897

32. https://blog.stackpuz.com/create-a-crud-api-with-fastapi/

33. https://github.com/wpcodevo/fastapi_sqlalchemy

34. https://shiftasia.com/community/build-a-simple-crud-api-using-fastapi-python-tutorial/

35. https://betterprogramming.pub/my-first-crud-app-with-fast-api-74ac190d2dcc

36. https://openf1.org

37. https://www.geeksforgeeks.org/python/fastapi-crud-operations/

38. https://www.youtube.com/watch?v=uGf2YTieHNw

39. https://github.com/LuisLuii/FastAPIQuickCRUD

40. https://github.com/br-g/openf1

41. https://dev.to/rock_win_c053fa5fb2399067/building-a-crud-application-with-fastapi-a-complete-guide-1897

42. https://blog.stackpuz.com/create-a-crud-api-with-fastapi/

43. https://github.com/wpcodevo/fastapi_sqlalchemy

44. https://shiftasia.com/community/build-a-simple-crud-api-using-fastapi-python-tutorial/

45. https://blog.stackpuz.com/create-a-crud-api-with-fastapi/

46. https://betterprogramming.pub/my-first-crud-app-with-fast-api-74ac190d2dcc

47. https://shiftasia.com/community/build-a-simple-crud-api-using-fastapi-python-tutorial/

48. https://shiftasia.com/community/build-a-simple-crud-api-using-fastapi-python-tutorial/

49. https://blog.stackpuz.com/create-a-crud-api-with-fastapi/

50. https://github.com/fastapi/fastapi/issues/2582

51. https://stackoverflow.com/questions/68505216/modulenotfounderror-no-module-named-app-routes

52. https://www.youtube.com/watch?v=gBilJFXundc

53. https://www.reddit.com/r/FastAPI/comments/1b7wooj/how_to_fix_the_modulenotfounderror_caused_by_the/

54. https://stackoverflow.com/questions/77114231/uvicorn-cant-find-modules

55. https://github.com/tiangolo/uvicorn-gunicorn-fastapi-docker/issues/225

56. https://github.com/Nuitka/Nuitka/issues/1063

57. https://github.com/fastapi/fastapi/discussions/6937

58. https://github.com/fastapi/fastapi/discussions/7449

59. https://sentry.io/answers/fastapi-docker-no-module-named-app-error/

60. https://stackoverflow.com/questions/76886981/importerror-cannot-import-name-base-from-partially-initialized-module-databa

61. https://github.com/fastapi/sqlmodel/issues/221

62. https://stackoverflow.com/questions/71119413/fastapi-error-cannot-import-name-fastapi-from-fastapi

63. https://github.com/fastapi-users/fastapi-users/issues/1077

64. https://www.codingeasypeasy.com/blog/fix-importerror-cannot-import-name-from-fastapi-a-comprehensive-guide

65. https://lightrun.com/answers/fastapi-users-fastapi-users-bug-importerror-cannot-import-name-sqlalchemybaseusertableuuid-from-fastapi_usersdb

66. https://stackoverflow.com/questions/79023460/handling-circular-imports-in-pydantic-models-with-fastapi

67. https://www.reddit.com/r/learnprogramming/comments/176eqfl/could_anyone_provide_some_inside_on_why_im/

68. https://github.com/sqlalchemy/sqlalchemy/discussions/9576

69. https://github.com/fastapi/fastapi/issues/1780

70. https://openf1.org/docs/

71. https://github.com/br-g/openf1/blob/main/README.md

72. https://mundobytes.com/en/What-is-the-openf1-api-for/

73. https://viktorsapozhok.github.io/fastapi-oauth2-postgres/

74. https://deepwiki.com/fastapi-practices/fastapi_best_architecture/7.3-crud-patterns-and-service-layer

75. https://github.com/br-g/openf1

76. https://www.juheapi.com/freeapis/openf1-car-data

77. https://github.com/br-g/openf1/discussions/66

78. https://github.com/bordanattila/OpenF1_tutorial

79. https://openf1.org/docs/

80. https://github.com/br-g/openf1/discussions/66

81. https://openf1.org

82. https://www.geeksforgeeks.org/python/making-http-requests-from-a-fastapi-application-to-an-external-api/

83. https://github.com/br-g/openf1

84. https://github.com/bordanattila/OpenF1_tutorial

85. https://openpublicapis.com/api/f1-api

86. https://stackoverflow.com/questions/63872924/how-can-i-send-an-http-request-from-my-fastapi-app-to-another-site-api

87. https://github.com/elarswa/openf1-sdk

88. https://github.com/lawrencebensaid/OpenF1-API/blob/main/README.md

89. https://www.geeksforgeeks.org/python/making-http-requests-from-a-fastapi-application-to-an-external-api/

90. https://stackoverflow.com/questions/63872924/how-can-i-send-an-http-request-from-my-fastapi-app-to-another-site-api

91. https://openf1.org/docs/

92. https://github.com/br-g/openf1/discussions/66

93. https://stackoverflow.com/questions/76886981/importerror-cannot-import-name-base-from-partially-initialized-module-databa